

```
[416]: #추천 의사(recommend_to_friend)는 해당 기업을 다른 사람에게 추천하고 싶은지를 가리킨다.

#변수들을 확인하고 추천 의사가 있는지 여부를 예측하는 의사결정나무 모델을 만드시오.

#1. 적절한 전처리를 진행하시오. (필요없는 변수 제거 포함) (학습용 데이터와 테스트용 데이터는 6:4로 분할할 것) o
#2. 최적의 나무 깊이를 찾고 학습시키시오. o
#3. 모델의 정확도를 제시하시오. o
#4. ROC 커브를 그리고 AUC 값을 제시하시오. o
#5. 변수 중요도 분석을 통하여 변수들의 중요도를 비교하시오. o
#6. 랜덤포레스트 모델로 모델을 학습시키고(최적 나무 개수 제시할 것) 정확도를 제시하시오.""
```

```
[417]: # 패키지 불러오기
import pandas as pd
import numpy as np
from sklearn import model_selection
from sklearn import metrics
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
import seaborn as sns
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestClassifier
```

```
[418]: # 데이터프레임으로 데이터 불러오기
df = pd.read_csv("data/dataset_MINI_EXAM.csv")
print(df.shape)
df.head()
```

```
(2008, 14)
[418]:
```

	Review_ID	Inudusty	Inudusty_level_2	Occupation	Experience	Pros	Cons	overall	advancement_rating	compensation_rating	worklife_balance_rating	culture_rating	management_rating	recommend_to_friend
0	2864282	IT/웹/통신	쇼핑몰/오픈마켓	IT/인터넷	관료적이고 비효율적인 업무는 비즈니스의 변화와 성장가운데 리스크 관리가 체계	관료적이고 비효율적인 업무는 비즈니스의 변화와 성장가운데 리스크 관리가 체계	체계 없고 체계가 없는 상태로 외형이	3	3	3	3	3		

```
[419]: print(df.columns)

Index(['Review_ID', 'Inudusty', 'Inudusty_level_2', 'Occupation', 'Experience',
      'Pros', 'Cons', 'overall', 'advancement_rating', 'compensation_rating',
      'worklife_balance_rating', 'culture_rating', 'management_rating',
      'recommend_to_friend'],
      dtype='object')

[420]: # 전처리
df = df.drop(['Inudusty', 'Inudusty_level_2', 'Review_ID', 'Experience', 'Pros', 'Cons'], axis=1)
```

```
[421]: df
```

```
[421]:
```

	Occupation	overall	advancement_rating	compensation_rating	worklife_balance_rating	culture_rating	management_rating	recommend_to_friend
0	IT/인터넷	3	3	3	3	1	1	Yes
1	인사/총무	2	4	4	3	3	2	No
2	경영/기획/컨설팅	4	4	3	1	4	1	Yes

3	마케팅/시장조사	4	4	4	4	4	4	Yes
4	디자인	5	5	5	5	5	5	Yes
...	...	...	...	...	...	...	...	...
2003	마케팅/시장조사	3	3	3	1	5	1	Yes
2004	IT/인터넷	5	4	4	5	2	5	Yes
2005	IT/인터넷	4	3	3	3	4	3	Yes
2006	영업/제휴	4	4	3	2	5	5	Yes
2007	영업/제휴	2	3	2	3	3	3	No

2008 rows × 8 columns

```
[422]: # 결측치 확인
df.isnull().sum()
```

```
[422]: Occupation      0
overall           0
advancement_rating 0
compensation_rating 0
worklife_balance_rating 0
culture_rating     0
management_rating  0
recommend_to_friend 0
dtype: int64
```

```
[423]: #df['Inudusty'].value_counts()
```

```
[424]: # 전처리: 권위점 변수로 변경
#df['Inudusty'] = df.Inudusty.map({'IT/웹/통신':0})
df['recommend_to_friend'] = df.recommend_to_friend.map({'Yes':0, 'No':1})
```

```
[425]: df = pd.get_dummies(df)
```

```
[426]: df
```

```
[426]:
```

	overall	advancement_rating	compensation_rating	worklife_balance_rating	culture_rating	management_rating	recommend_to_friend	Occupation_IT/인터넷	Occupation_경영/기술컨설팅
0	3	3	3	3	1	1	0	True	False
1	2	4	4	3	3	2	1	False	False
2	4	4	3	1	4	1	0	False	True
3	4	4	4	4	4	4	0	False	False
4	5	5	5	5	5	5	0	False	False
...	...	...	...	...	...	...	...	...	...
2003	3	3	3	1	5	1	0	False	False
2004	5	4	4	5	2	5	0	True	False
2005	4	3	3	3	4	3	0	True	False
2006	4	4	3	2	5	5	0	False	False
2007	2	3	2	3	3	3	1	False	False

2008 rows × 25 columns

```
[427]: #recommend_to_friend 열 맨 오른쪽으로 이동
cols = [col for col in df.columns if col != 'recommend_to_friend']
df = df[cols + ['recommend_to_friend']]
```

```
[428]: # Input 변수(X)와 Output 변수(y) 구분
X = np.array(df.iloc[:, :-1])
y = np.array(df['recommend_to_friend'])
```

```
[429]: # 트레이닝 셋트와 테스트 셋트로 데이터 구분(6:4)
from sklearn.model_selection import train_test_split

# random_state는 반복적으로 같은 결과를 내기 위해서 설정
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=0)

print("X_train 크기:", X_train.shape)
print("y_train 크기:", y_train.shape)
print("X_test 크기:", X_test.shape)
print("y_test 크기:", y_test.shape)

X_train 크기: (1204, 24)
y_train 크기: (1204,)
X_test 크기: (804, 24)
y_test 크기: (804,)
```

```
[430]: train_scores, test_scores = list(), list()
# train_score와 test_score의 정확도를 저장 장소
# train_score는 훈련데이터에 특화될수록 증가
# test_score는 테스트데이터에 일반화될수록 증가
```

```
[431]: # 의사결정나무모델에 데이터 적합(fitting)
from sklearn.tree import DecisionTreeClassifier

#의사결정나무 모델의 depth 조절(1부터 19까지)
for i in range(1,20):
    DT1 = DecisionTreeClassifier(max_depth=i)
    #깊이를 증가시키면서 의사결정나무 모델 적합
    DT1.fit(X_train, y_train)

    #training dataset 정확도 평가
    train_att = DT1.predict(X_train)
    train_acc = accuracy_score(y_train, train_att)
    train_scores.append(train_acc)

    #test dataset 정확도 평가
    test_att = DT1.predict(X_test)
    test_acc = accuracy_score(y_test, test_att)
    test_scores.append(test_acc)

    #train 정확도와 test 정확도를 depth 개수에 따른 정확도 확인
    print('>%d, train: %.3f, test: %.3f' % (i, train_acc, test_acc))
```

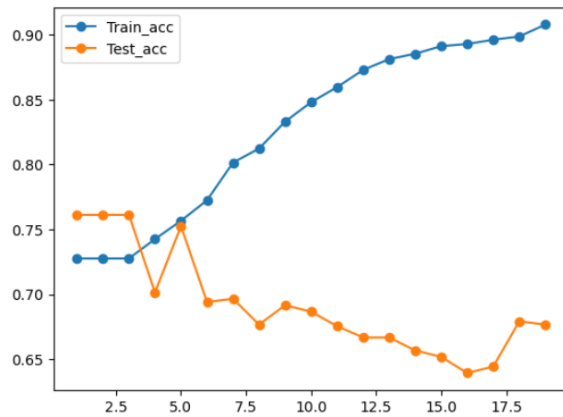
```
>1, train: 0.728, test: 0.761
>2, train: 0.728, test: 0.761
>3, train: 0.728, test: 0.761
>4, train: 0.743, test: 0.701
>5, train: 0.757, test: 0.752
>6, train: 0.772, test: 0.694
>7, train: 0.801, test: 0.697
>8, train: 0.812, test: 0.677
>9, train: 0.833, test: 0.692
>10, train: 0.848, test: 0.687
>11, train: 0.860, test: 0.675
>12, train: 0.873, test: 0.667
>13, train: 0.881, test: 0.667
>14, train: 0.885, test: 0.657
>15, train: 0.891, test: 0.652
>19, train: 0.908, test: 0.677
```

[432]: #그래프 그리기

```
from matplotlib import pyplot

pyplot.plot(range(1,20), train_scores, '-o', label='Train_acc')
pyplot.plot(range(1,20), test_scores, '-o', label='Test_acc')
pyplot.legend()
pyplot.show()

#최적의 의사결정나무 깊이는? 5
```



```
[433]: #의사결정나무의 깊이를 설정하여 재학습
#파라미터(max_depth)에 최적값이 갖 부여

tree = DecisionTreeClassifier(random_state=0, max_depth=5) #max_depth 5로 지정
tree.fit(X_train, y_train)
```

```
[433]: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=5, random_state=0)
```

```
[434]: #변수의 중요도(y값 결정에 대한 기여 정도)를 변수로 수치화하여 보여줌

tree.feature_importances_
```

```
[434]: array([[0.56486921, 0.05768165, 0.08079347, 0.04078317, 0.05097584,
0.16706976, 0.00740013, 0.00405233, 0.00613989, 0.00440633,
0.00656152, 0.00265066, 0. , 0. , 0. ,
0. , 0. , 0. , 0. , 0. ,
0. , 0. , 0.00661604, 0. ]])
```

```
[435]: pd.DataFrame(tree.feature_importances_.reshape(1,-1), index=['feature_importance'])
```

```
[435]:
```

	0	1	2	3	4	5	6	7	8	9	...	14	15	16	17	18	19	20	21	22
feature_importance	0.564869	0.057682	0.080793	0.040783	0.050976	0.16707	0.0074	0.004052	0.00614	0.004406	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.006616

1 rows x 24 columns

```
[436]: df.columns

pd.DataFrame(tree.feature_importances_.reshape(1,-1),
              columns=df.columns[1:], index=['feature_importance'])

#보기 좋게 칼럼명을 바꿈
```

```
[436]:
```

	advancement_rating	compensation_rating	worklife_balance_rating	culture_rating	management_rating	Occupation_IT/ 인터넷	Occupation_ 경영/기획/ 컨설팅	Occupation_ 교육
feature_importance	0.564869	0.057682	0.080793	0.040783	0.050976	0.16707	0.0074	0.004052

1 rows x 24 columns

```
[437]: #변수중요도를 데이터프레임으로 만들기

importance_df = pd.DataFrame({'feature': df.columns[1:], 'importance': tree.feature_importances_})
importance_df
```

```
[437]:
```

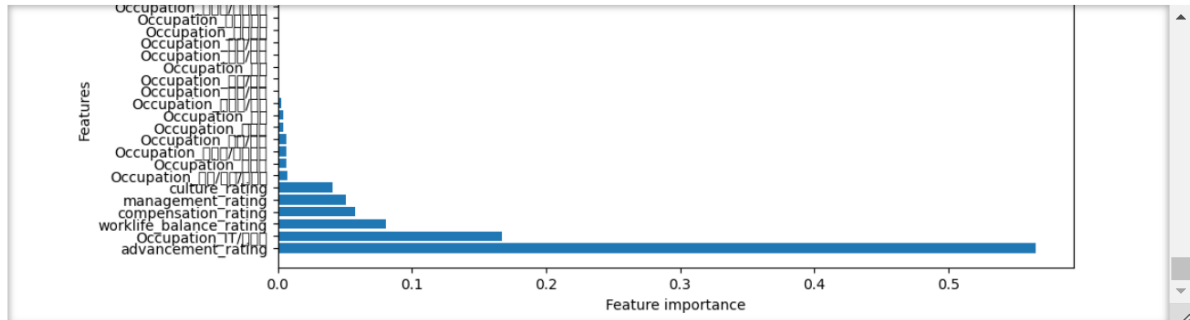
	feature	importance
0	advancement_rating	0.564869
1	compensation_rating	0.057682
2	worklife_balance_rating	0.080793
3	culture_rating	0.040783
4	management_rating	0.050976
5	Occupation_IT/인터넷	0.167070
6	Occupation_경영/기획/컨설팅	0.007400
7	Occupation_교육	0.004052
8	Occupation_금융/재무	0.006140
9	Occupation_디자인	0.004406
10	Occupation 마케팅/시장조사	0.006562

22	Occupation_전문직	0.006616
23	recommend_to_friend	0.000000

```
[438]: #중요도에 따라 내림차순으로 정렬
importance_df = importance_df.sort_values(by='importance', ascending=False)

import matplotlib.pyplot as plt
#막대 그래프로 시각화
plt.figure(figsize=(10,4))
plt.barh(importance_df['feature'], importance_df['importance'])
plt.xlabel('Feature importance')
plt.ylabel('Features')
plt.title('Feature importance from Decision Tree Model')
plt.show()

#한글을 지원하지 않아 글자가 완전히 보이지 않지만, advancement_rating이 가장 중요하다는 것을 알 수 있음
```



```
[439]: #모델 성능 평가
temp_y_pred = tree.predict(X_test)
#Training값을 X_test에서 불러옴
print('예측값\n', temp_y_pred)
print('실제값\n', y_test)

예측값
[0 0 0 0 1 1 0 0 0 0 1 0 0 1 0 0 1 0 0 1 1 1 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 1
 0 1 0 0 0 0 0 1 0 0 0 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 0 0 0 1
 1 0 1 1 1 1 1 0 0 0 1 1 0 1 0 1 1 0 0 0 1 1 0 0 1 1 0 1 1 0 1 1 0 0 1 1 1
 1 0 1 0 1 0 1 1 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1
 1 1 0 0 1 1 0 0 1 0 1 1 0 0 0 1 0 1 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 0
 1 0 1 0 1 0 1 0 1 0 1 0 1 1 1 1 0 0 1 1 1 1 0 1 0 1 0 0 1 1 1 0 1 0 1 1
 1 0 1 1 1 0 0 0 1 1 0 0 1 0 0 1 1 1 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0 1 0 0 0
 1 0 1 1 1 1 0 1 1 1 1 1 0 0 1 1 1 1 1 1 1 0 0 1 1 0 1 1 1 1 0 1 1 0 1 1
 1 1 0 1 1 1 0 0 1 1 1 1 1 1 1 0 0 1 0 1 1 1 0 0 0 1 1 1 1 0 1 1 0 1 1 0
 0 0 1 1 1 0 1 1 0 0 1 0 1 1 0 0 0 0 1 0 0 0 1 1 1 1 0 0 0 0 1 1 0 1 0 0 1 1 0
 1 1 1 1 0 0 0 0 0 0 0 1 1 1 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 1 1 0 0 1 1 0
 1 1 0 1 1 1 1 1 1 1 1 0 0 1 0 1 0 0 0 1 0 0 0 1 1 1 1 1 1 1 1 0 1 0 0 0
 0 0 1 0 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 0 0 1
 1 1 0 0 1 0 1 0 0 1 0 1 0 1 0 1 0 0 0 1 1 0 0 1 0 0 1 1 1 1 0 1 1 1 1 0 1
 1 1 0 1 0 1 1 0 0 0 1 1 0 0 0 1 0 0 1 1 1 0 0 0 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 0 1 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 0 1 0 0 1 1 1 1 1 1 1 1 1 0 1
 1 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 1 1 0 0 1 0 0 1 1
 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 0 1 1 0 1 0 0 1 1 0 1 1 0
 0 1 1 0 1 0 1 1 1 0 0 0 1 1 0 1 0 1 0 1 0 1 1 1 1 0 1 0]

실제값
[0 0 0 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 0 1 1 1 1 0 0 1 0 0 0 1 1 1 1 0 1 1
 0 1 0 0 0 0 0 1 1 0 0 1 0 1 1 1 1 0 0 0 0 1 0 1 1 1 0 0 1 1 0 1 0 0 1
 1 0 1 1 1 0 1 0 0 0 1 1 0 1 0 1 1 0 0 1 1 1 1 0 0 1 0 0 1 1 0 1 1 0 0 1 1 1
 1 0 1 0 1 0 1 1 0 1 1 0 1 0 0 1 0 0 1 1 0 0 1 1 1 1 1 1 1 0 0 1 1 1 1 0 1 1
 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 1 0 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 1 1 0
 1 0 1 0 0 1 0 0 1 1 1 0 1 1 0 1 1 1 1 1 0 0 0 1 1 1 0 1 1 1 0 0 0 0 1 0 0 0
 1 0 1 1 1 1 0 1 1 1 1 1 0 0 0 1 0 1 0 0 0 1 1 0 1 1 1 0 1 1 1 1 1 1 1 0 1
 1 0 0 0 1 1 1 1 0 1 0 1 0 1 0 1 0 0 1 0 1 0 0 0 1 1 0 1 0 1 1 0 0 0 1 1
 1 0 0 0 1 1 0 1 0 1 0 1 0 1 1 1 0 0 0 1 0 1 0 0 1 0 1 0 0 0 1 0 1 1 1 0 1
 1 0 0 1 1 0 0 0 0 1 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 1 1 1 1 1 1 0 0 0 1
 1 1 0 1 0 1 1 0 0 0 1 0 0 1 1 1 0 1 1 1 0 0 1 1 1 1 1 1 0 0 1 0 0 0 1 1
 1 1 0 1 0 1 0 0 1 1 0 0 0 1 0 0 1 1 0 0 1 0 1 0 0 0 1 1 1 1 1 1 0 1 0 1 0
 1 0 0 0 0 1 0 0 0 0 1 1 1 0 0 0 0 1 1 0 0 0 1 0 0 0 0 1 1 1 0 1 1 0 0 0 0
 1 0 1 1 0 0 0 0 0 1 0 1 1 1 1 0 1 0 1 0 1 0 1 1 1 1 1 0 1 0 1 1 1 1 0 1 1
 1 0 0 1 0 1 1 0 1 1 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1 1 0 0 1 0 0 0 1 1 0 1 0
 0 0 0 1 0 1 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1
 1 1 1 1 0 1 0 1 0 0 0 1 0 1 1 1 1 0 0 0 1 0 0 1 0 1 0 1 1 1 0 1 1 1 0 0 1
 0 0 1 1 1 0 1 1 0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 1 1 1 1 0 0 1 1 1 0 1 1 1
 1 0 1 0 1 0 1 0 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 1 1 1 1 1 1 1 1 0 0 1
 1 0 1 1 1 1 1 1 1 0 1 0 0 1 1 1 0 0 0 1 1 1 0 0 1 1 0 0 1 1 0 0 1 0 1 1 1
 0 1 1 1 0 1 0 0 0 1 1 0 1 0 0 1 1 0 1 1 0 0 0 0 0 1 1 0 1 1 1 1 0 0 1 0 0
 0 1 1 0 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 1 0 1 1 0 1 0]

[440]: # 정확도 계산
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

temp_acc = accuracy_score(y_test, temp_y_pred)
# y_test와 temp_y_pred로 예측한 값을 직렬화하여 temp_acc를 추출함.

print('정확도: ', format(temp_acc))

정확도: 0.7549751243781094
```

```
[441]: tree.predict_proba(X_test)[: ,1]
```

```
[441]: array([0.11214953, 0.11214953, 0.11214953, 0.45238095, 0.51320755,
        0.55555556, 0.11214953, 0.11214953, 0.11214953, 0.04878049,
        0.51320755, 0.11214953, 0.33333333, 0.97927461, 0.45238095,
        0.33333333, 0.92857143, 0.04878049, 0.45238095, 0.56666667,
        0.51320755, 0.88405797, 0.66666667, 0.51320755, 0.51320755,
        0.71428571, 0.51320755, 0.11214953, 0.51320755, 0.11214953,
        0.51320755, 0.51320755, 0.51320755, 0.97927461, 0.11214953,
        0.27272727, 0.51320755, 0.07692308, 0.97927461, 0.11214953,
        0.11214953, 0.45238095, 0.11214953, 0.11214953, 0.93333333,
        0.04878049, 0.04878049, 0.04878049, 0.88405797, 0.07692308,
        0.97927461, 0.51320755, 0.51320755, 0.97927461, 0. ,
        0.51320755, 0.51320755, 0.51320755, 0.51320755, 0.11214953,
        0.56666667, 0.97927461, 0.51320755, 0.97927461, 0.97927461,
        0.51320755, 0.51320755, 0.97927461, 0.92857143, 0.04878049,
        0.11214953, 0.11214953, 0.27272727, 0.92857143, 0.51320755,
        0.45238095, 0.51320755, 0.51320755, 0.93333333, 0.97927461,
        0.51320755, 0.11214953, 0.11214953, 0.45238095, 0.71428571,
        0.75, 0.07692308, 0.97927461, 0.45238095, 0.51320755,
```

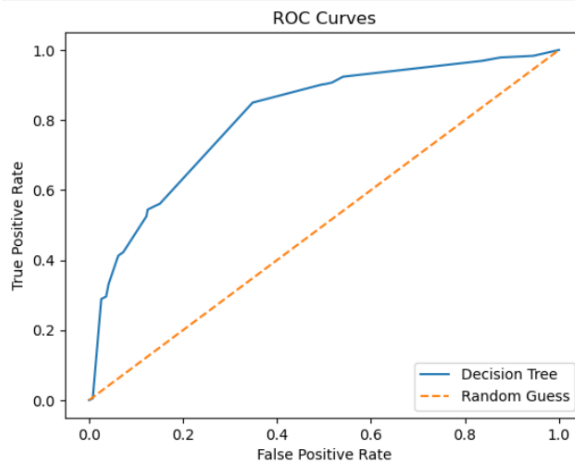
```
[442]: from sklearn.metrics import roc_curve
```

```
fpr, tpr, thresholds = roc_curve(y_test, tree.predict_proba(X_test)[: ,1])
```

```
[443]: #ROC curve 그래프 그리기
```

```
plt.plot(fpr, tpr, '-', ms=2, label="Decision Tree")
plt.plot([0,1], [0,1], '--', label="Random Guess")
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend(loc="lower right")
plt.show()
```



```
[444]: #AUC 계산
```

```
from sklearn import metrics
roc_auc = metrics.auc(fpr, tpr)
print('Area Under Curve : %0.2f' % roc_auc)
```

```
Area Under Curve : 0.81
```

```
[445]: scores = []
for i in range(10,300,5):
    RF = RandomForestClassifier(n_estimators=i, random_state=0)
    RF.fit(X_train, y_train)

    att = RF.predict(X_test)
    acc = accuracy_score(y_test, att)
    scores.append(acc)

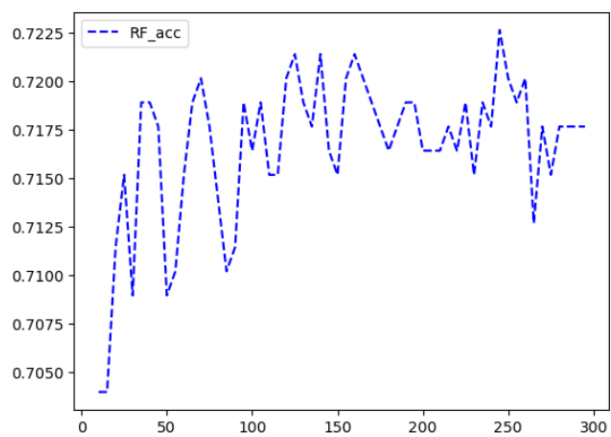
    print('>%d, acc: %.3f:' % (i, acc))
```

```
>10, acc: 0.704:
>15, acc: 0.704:
>20, acc: 0.711:
>25, acc: 0.715:
>30, acc: 0.709:
>35, acc: 0.719:
>40, acc: 0.719:
>45, acc: 0.718:
>50, acc: 0.709:
>55, acc: 0.710:
>60, acc: 0.715:
>65, acc: 0.719:
>70, acc: 0.720:
>75, acc: 0.718:
>80, acc: 0.714:
>85, acc: 0.710:
>90, acc: 0.711:
>95, acc: 0.719:
>100, acc: 0.716:
>105, acc: 0.719:
>110, acc: 0.715:
>115, acc: 0.715:
>120, acc: 0.720:
>125, acc: 0.721:
```

```
[446]: import matplotlib.pyplot as plt

plt.plot(range(10, 10 + 5 * len(scores), 5), scores, 'b--', label='RF_acc')
plt.legend()
```

```
[446]: <matplotlib.legend.Legend at 0x2b931e14ec0>
```



```
[447]: # scores에서 가장 높은 정확도의 인덱스(최적의 나무 개수 확인)
optimal_trees = np.argmax(scores) * 5 + 10 # i가 10부터 시작하고, 5씩 증가했으므로 계산
optimal_acc = max(scores) # 최고 정확도

print(f"최적의 나무 개수: {optimal_trees}, 정확도: {optimal_acc:.3f}")

최적의 나무 개수: 245, 정확도: 0.723
```

```
[448]: # 의사결정나무모델에 데이터 적합(fitting)
# 랜덤 포레스트
# n_estimators = 245로 새로 적합

from sklearn.ensemble import RandomForestClassifier

RF = RandomForestClassifier(n_estimators=245, random_state=0)
#n_estimators: 사용할 tree의 개수.
```

```
[449]: # 모델 적합
RF.fit(X_train, y_train)
```

```
[449]: RandomForestClassifier
RandomForestClassifier(n_estimators=245, random_state=0)
```

```
[450]: from sklearn.metrics import roc_curve

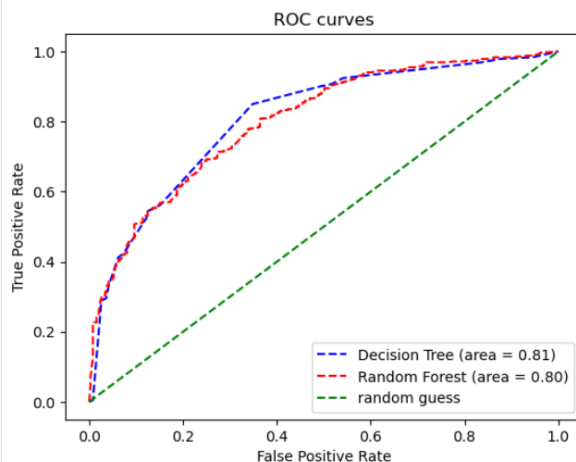
fpr1, tpr1, thresholds1 = roc_curve(y_test, tree.predict_proba(X_test)[:,:1])
fpr2, tpr2, thresholds2 = roc_curve(y_test, RF.predict_proba(X_test)[:,:1])
```

```
[451]: roc_auc_DT = metrics.auc(fpr1, tpr1)
roc_auc_RF = metrics.auc(fpr2, tpr2)
```

```
[452]: # 연립에 AUC 값 출력
plt.plot(fpr1, tpr1, 'b--', label = 'Decision Tree (area = %0.2f)' % roc_auc_DT)
plt.plot(fpr2, tpr2, 'r--', label = 'Random Forest (area = %0.2f)' % roc_auc_RF)
plt.plot([0,1], [0,1], 'g--', label = 'random guess')

plt.title('ROC curves')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc = 'lower right')
```

```
[452]: <matplotlib.legend.Legend at 0x2b931924890>
```



```
[453]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 성능 평가
temp_y_pred_rf = RF.predict(X_test)

print('accuracy: ', accuracy_score(y_test, temp_y_pred_rf))
print('precision: ', precision_score(y_test, temp_y_pred_rf))
print('recall: ', recall_score(y_test, temp_y_pred_rf))
print('f1: ', f1_score(y_test, temp_y_pred_rf))

accuracy: 0.722636815920398
precision: 0.7158590308370044
recall: 0.7756563245823389
f1: 0.7445589919816724
```

```
[454]: #60211356 정유진
```


1. 적절한 전처리를 진행하시오. (필요없는 변수 제거 포함)
(학습용 데이터와 테스트용 데이터는 6:4로 분할할 것)

2. 최적의 나무 깊이를 찾고 학습시키시오.

3. 모델의 정확도를 제시하시오.

-> 약 0.755이다.

4. ROC 커브를 그리고 AUC 값을 제시하시오.

5. 변수 중요도 분석을 통하여 변수들의 중요도를 비교하시오.

-> advancement_rate 가 가장 높은 것을 볼 수 있다.

6. 랜덤포레스트 모델로 모델을 학습시키고(최적 나무 개수 제시할 것) 정확도를 제시하시오.

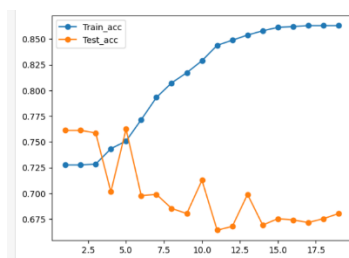
-> 최적의 나무 개수는 245개이다.

-> 정확도는 약 0.723이다.

후기

랜덤포레스트 모델을 사용하고 오히려 정확도가 하락하였다. 전처리하는 과정에서 정확도를 하락시킬 원인이 있었다고 생각한다.

또한, 처음에 전처리를 하였을 때에 occupation을 제거하였는데, 최적의 의사결정나무 깊이를 구할 때



이런 모양의 그래프가 나왔다. 이는 의사결정나무의 깊이가 5일 때, 과적합할 가능성을 보여 전처리를 다시 하기로 결정하였다. 다시 전처리를 할 때에는 occupation을 제거하지 않고, 원핫인코딩을 사용하여 분류하였고, 깊이가 5일 때 과적합이 일어나지 않도록 조정하였다.

여러 과정을 거치면서 의사결정나무모델과 랜덤포레스트 모델에 대한 이해도를 높일 수 있었다.